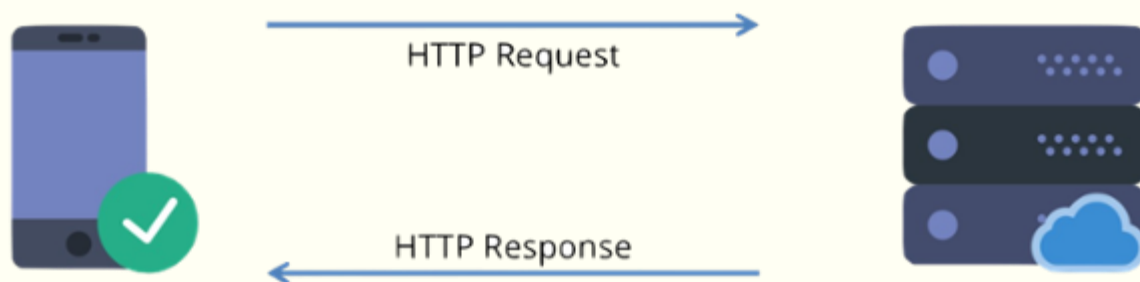


АСИНХРОННІ НТТР-ЗАПИТИ З ВИКОРИСТАННЯМ FETCH API. ОБРОБКА ПОМИЛОК ПІД ЧАС НТТР-ВЗАЄМОДІЇ

Питання 2.2.

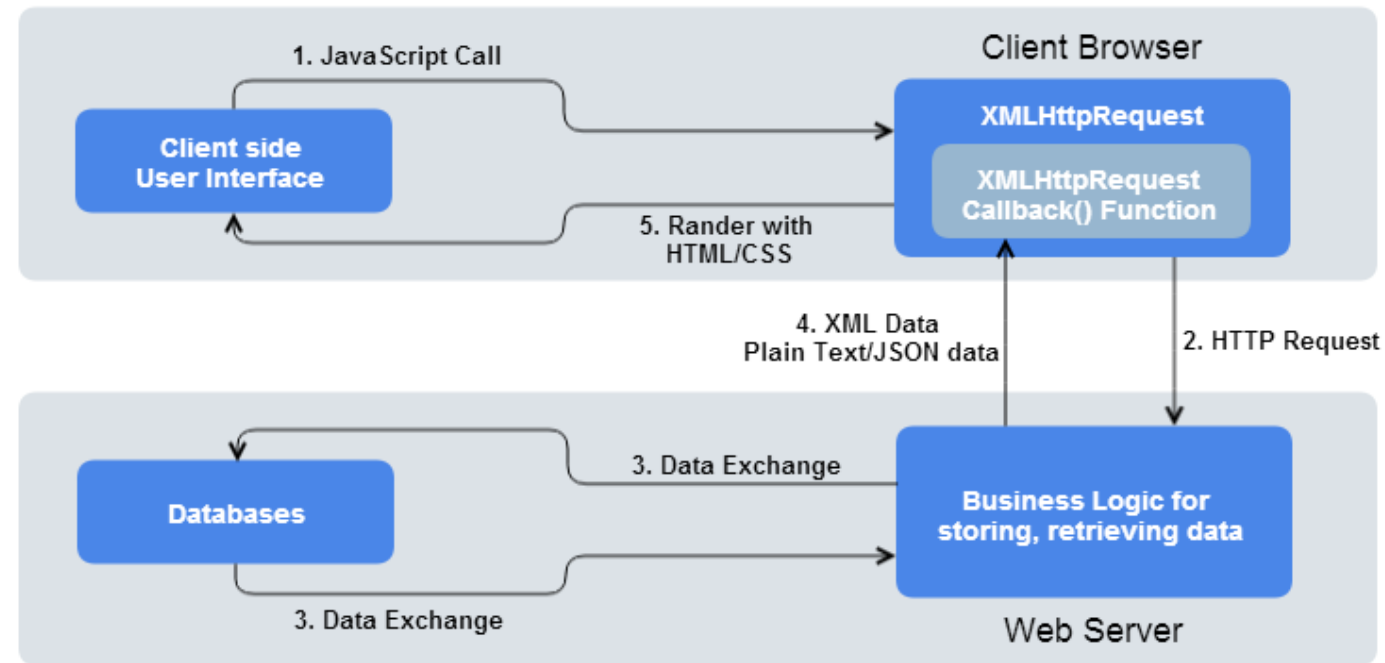
Фетчинг даних у цілому

- Класичне завантаження сторінок в Інтернеті було досить простим: ви надсилали запит щодо вебсайту на сервер, і якщо все йшло так, будь-які ресурси, що складали сторінку, завантажувалися та відображалися на вашому комп'ютері чи пристрої.



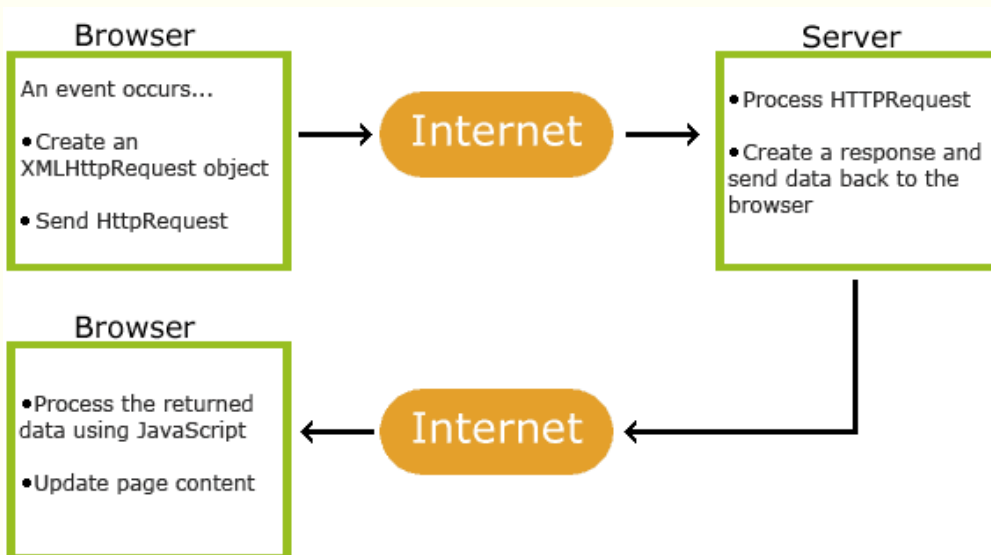
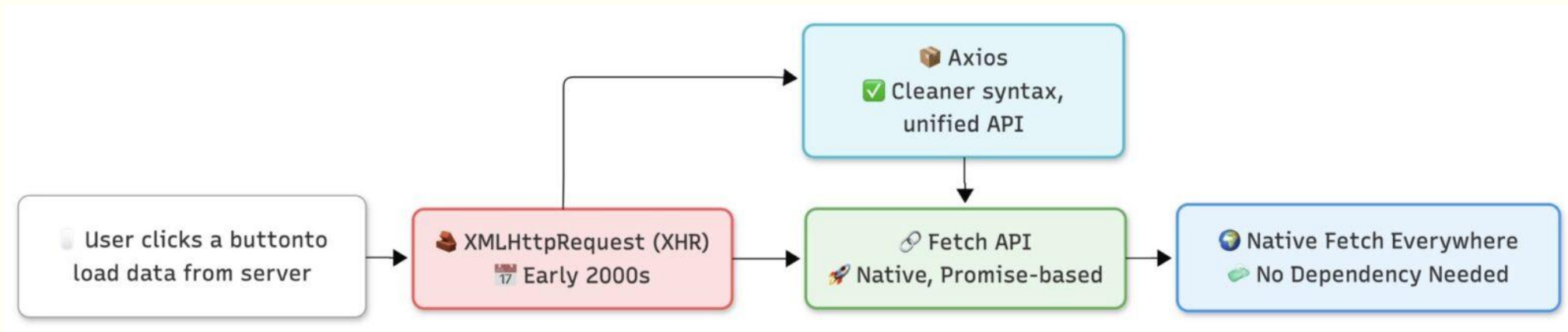
- Так вебсайти працювали роками. Однак у цьому було кілька недоліків...
- Щоразу, коли ви хотіли оновити будь-яку частину сторінки, наприклад, щоб відобразити новий набір продуктів або надіслати форму, вся сторінка мала завантажуватися знову.
- Така практика вимагає багато ресурсів і призводить до поганого UX, особливо коли сторінки зростають і стають складнішими.

Фетчинг даних у цілому



- Цю проблему було вирішено, оскільки були створені технології, які дозволили вебсторінкам запитувати невеликі фрагменти даних (такі як HTML, XML, JSON або звичайний текст) та відображати їх лише за потреби.
 - Технології XMLHttpRequest та Fetch API дозволяють вебсторінкам безпосередньо обробляти HTTP-запити для певних ресурсів, доступних на сервері, та формувати отримані дані за потреби перед їх відображенням.
 - Ця техніка називається AJAX (Asynchronous JavaScript and XML).
 - Концептуально, AJAX передбачає використання веб-API як проксі-сервера для більш інтелектуального запиту даних, а не просто перезавантаження всієї сторінки браузером.

Коротка історія засобів фетчингу в JavaScript



- XMLHttpRequest (XHR) був представлений Microsoft ще на початку 2000-х років, а пізніше прийнятий іншими браузерами.
 - Він був рушієм ранніх AJAX-запитів і революціонував спосіб нашої взаємодії з вебсерверами.
 - XHR дозволив асинхронно отримувати дані, але він не був розроблений з урахуванням сучасних шаблонів JavaScript. Його залежність від слухачів подій та ручного відстеження стану (наприклад `readyState`, кодів стану) може призвести до складніших та важчих у підтримці кодових баз.
 - Потім з'явився більш елегантний Fetch API на основі Promise, представлений як частина сучасного стандарту JavaScript.

AJAX = Asynchronous JavaScript And XML

Порівняння синтаксису XMLHttpRequest та Fetch API

Приклад XMLHttpRequest:

```
const xhr = new XMLHttpRequest();
xhr.open("GET", "/api/data");
xhr.onload = () => {
  if (xhr.status === 200) {
    console.log(xhr.responseText);
  }
};
xhr.send();
```

Приклад Fetch API:

```
fetch("/api/data")
  .then(res => res.text())
  .then(data => console.log(data));
```

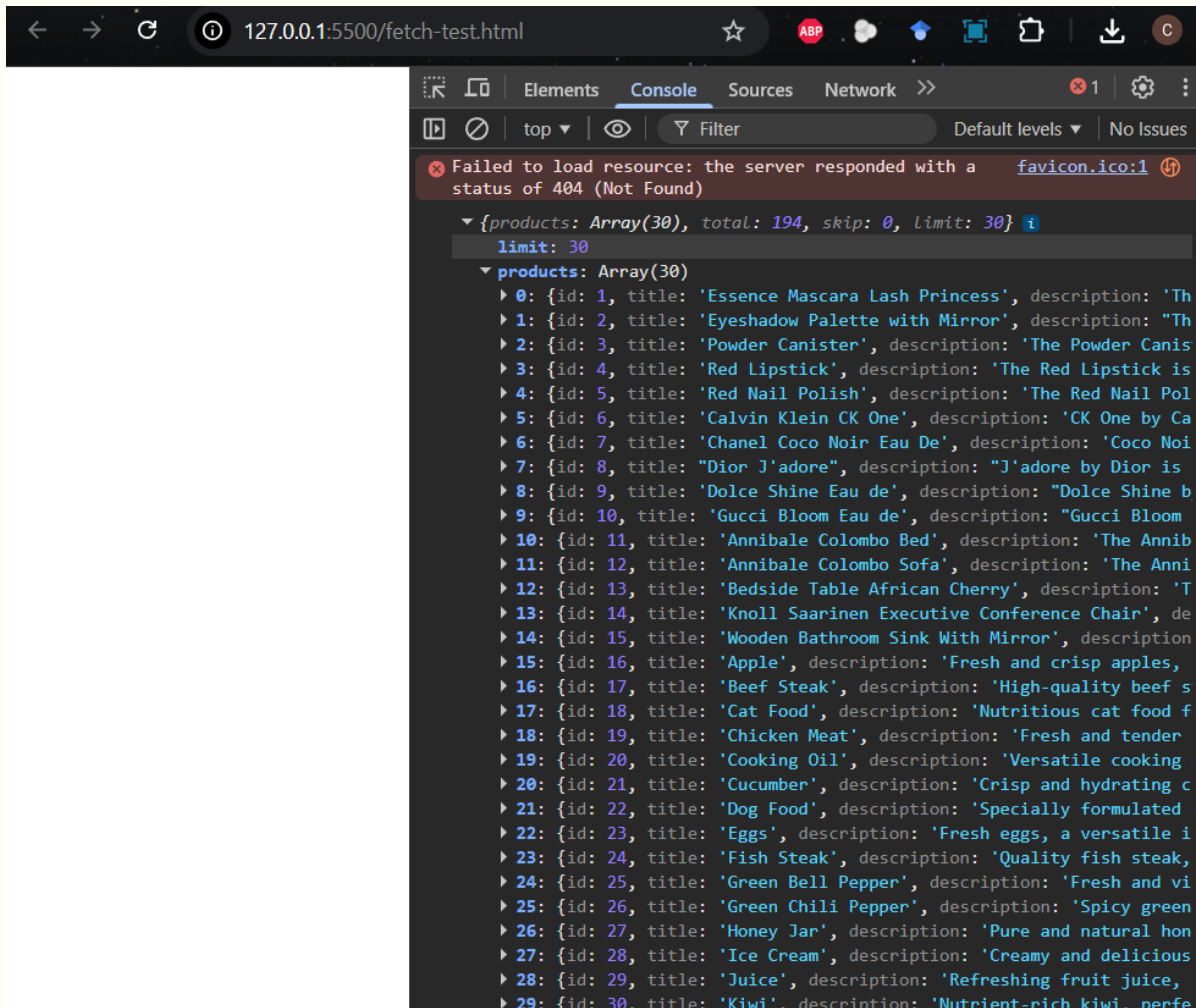
- Справа не лише в синтаксисі, `fetch()` також пропонує:
 - об'єкти `Request/Response` для кращого контролю та більшої гнучкості;
 - підтримку потоків для ефективної обробки великих обсягів даних;
 - `AbortController` для скасування запитів під час виконання;
 - покращену підтримку CORS «з коробки»;
 - послідовну обробку помилок, на відміну від `XHR`, яка вимагає додаткового шаблону для виявлення помилок;
 - підтримку сучасних типів контенту, таких як `JSON`, `FormData` та `Blob`.
- У контексті налагодження, `fetch()` краще інтегрується із сучасними інструментами розробника.
 - Легше відстежувати ланцюжок `Promise` та визначати помилки, ніж вручну відстежувати події та стани `XHR`.

Коротка історія засобів фетчингу в JavaScript

- Перш ніж `fetch()` став широко підтримуватися, як у браузерах, так і в середовищах, на зразок Node.js, розробники покладалися на сторонні бібліотеки для абстрагування та стандартизації мережевих запитів.
 - Такі інструменти, як `axios`, `superagent` і навіть `$.ajax` від jQuery, заповнили прогалину зручними API та узгодженою кросбраузерною поведінкою.
 - Ці бібліотеки додали такі функції, як скасування запитів, автоматичний розбір JSON, тайм-аути та чистіший синтаксис задовго до того, як `fetch` став готовим для прайм-тайму.
 - Але тепер, коли `fetch` є нативним майже у всіх основних середовищах (включаючи Node 18+), потреба в зовнішніх залежностях зменшується. Якщо ви створюєте сучасний JavaScript-додаток і прагнете зробити його легким, `fetch` ідеально вписується в принцип «vanilla first».
- Слід пам'ятати, що XMLHttpRequest все ще широко підтримується та може знадобитися у застарілих кодових базах або певних середовищах, де `fetch` не поліфілізований.
 - `fetch` не відхиляє повідомлення у випадках помилок HTTP (наприклад, 404 або 500): все одно потрібно перевірити `res.ok` вручну та відповідно обробляти ці випадки.

```
fetch("/api/missing")
  .then(res => {
    if (!res.ok) throw new Error("Request failed");
    return res.json();
  })
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Перший GET-запит



- `fetch()`— це вбудована функція браузера , яка дозволяє робити HTTP-запити.
 - Вона повертає Promise, який зрештою перетворюється на об'єкт Response.
 - Її реалізація написана мовою нижчого рівня (наприклад, C++ або Rust) всередині рушія браузера, тобто обробляється браузером внутрішньо.
- Скористаємося безкоштовним публічним API під назвою `dummyjson.com`.
 - Відповідь має тіло (де зберігаються дані) та заголовки.
 - Але ви не можете безпосередньо побачити тіло даних – це читабельний потік (readable stream).
 - Щоб витягти дані, викликається метод `.json()`

```
<!doctype html>
<meta charset="utf-8" />
<script>
  fetch("https://dummyjson.com/products")
    .then(r => r.json())
    .then(console.log)
    .catch(console.error);
</script>
```

```

<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <title>Fetch Demo</title>
</head>
<body>

  <button id="loadData">Завантажити дані</button>
  <pre id="output"></pre>

  <script>
    const btn = document.getElementById("loadData");
    const output = document.getElementById("output");

    btn.addEventListener("click", () => {
      fetch("https://dummyjson.com/products")
        .then(res => res.json())
        .then(data => {
          output.textContent = JSON.stringify(data.products, null, 2);
        })
        .catch(err => {
          console.error("Error:", err);
          output.textContent = "Помилка завантаження";
        });
    });
  </script>

</body>
</html>

```

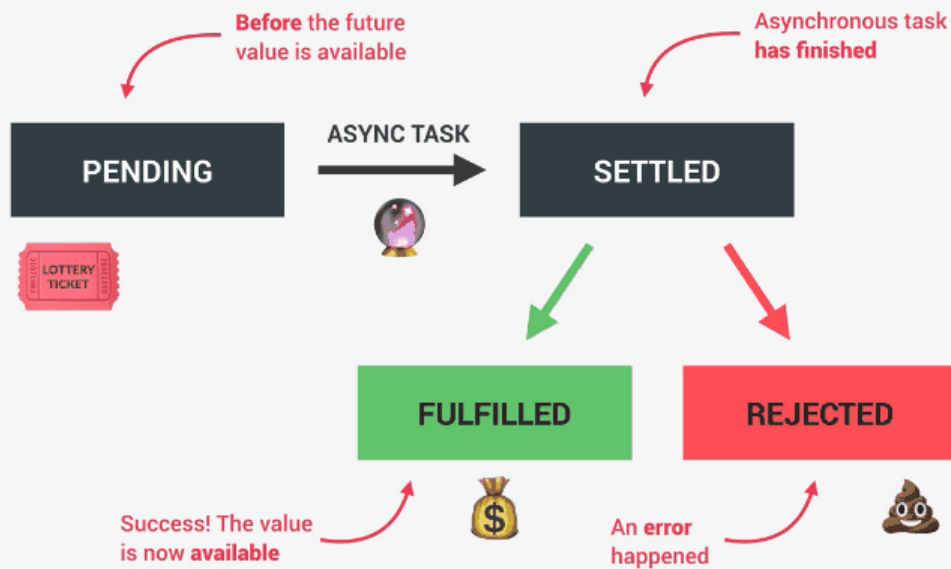
Перший GET-запит

- Підключимо дані до нашого HTML

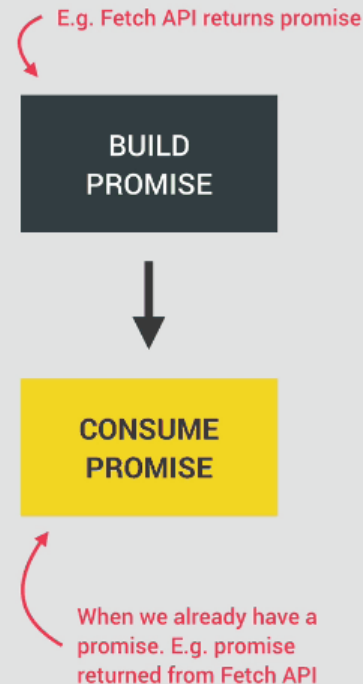


Проміси та Fetch API

THE PROMISE LIFECYCLE



👉 We are able **handle** these different states in our code!



- Проміс схожий на лотерейний квиток.

- Купівля квитка = купівлі обіцянки отримати гроші в майбутньому, якщо вгадаєш правильний результат.
- Функція `fetch()` будує проміс та повертає його для використання – нам не потрібно самостійно його створювати.
- Стрілкові функції в блоках `.then()` дають змогу не писати `return` явно:

```
fetch("https://dummyjson.com/products")  
  .then(res => {  
    return res.json();  
  })  
  .then(data => console.log(data));
```

- Fetch підтримує HTTP-методи GET, POST, PUT, DELETE та PATCH.

Синтаксис async/await

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <title>Fetch Demo</title>
</head>
<body>

  <button id="loadData">Завантажити дані</button>
  <pre id="output"></pre>

  <script>
    const btn = document.getElementById("loadData");
    const output = document.getElementById("output");

    btn.addEventListener("click", () => {
      fetch("https://dummyjson.com/products")
        .then(res => res.json())
        .then(data => {
          output.textContent = JSON.stringify(data.products, null, 2);
        })
        .catch(err => {
          console.error("Error:", err);
          output.textContent = "Помилка завантаження";
        });
    });
  </script>

</body>
</html>
```

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <title>Fetch Async/Await Demo</title>
</head>
<body>

  <button id="loadData">Завантажити дані</button>
  <pre id="output"></pre>

  <script>
    const btn = document.getElementById("loadData");
    const output = document.getElementById("output");

    btn.addEventListener("click", async () => {
      try {
        const res = await fetch("https://dummyjson.com/products");
        const data = await res.json();

        output.textContent = JSON.stringify(data.products, null, 2);
      } catch (err) {
        console.error("Error:", err);
        output.textContent = "Помилка завантаження";
      }
    });
  </script>

</body>
</html>
```

```

<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <title>POST (then)</title>
</head>
<body>
  <button id="send">Відправити POST</button>
  <pre id="output"></pre>

  <script>
const btn = document.getElementById("send");
const output = document.getElementById("output");

btn.addEventListener("click", () => {
  fetch("https://dummyjson.com/products/add", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      title: "BMW Pencil",
      category: "stationery"
    })
  })
  .then(res => res.json())
  .then(data => {
    console.log("Created:", data);
    output.textContent = JSON.stringify(data, null, 2);
  })
  .catch(err => {
    console.error("Error:", err);
    output.textContent = "Помилка запиту";
  });
});
</script>

</body>
</html>

```

Надсилання POST-запиту

The screenshot displays a web browser interface with a button labeled "Відправити POST". Below the button, a JSON response is shown in a light blue box:

```
{
  "id": 195,
  "title": "BMW Pencil",
  "category": "stationery"
}
```

The browser's developer tools are open, showing the Network tab. A request named "add" is selected, and the Headers sub-tab is active. The request details are as follows:

Name	Value
Request URL	https://dummyjson.com/products/add
Request Method	POST
Status Code	201 Created
Remote Address	104.21.61.23:443
Referrer Policy	strict-origin-when-cross-origin

```
<!DOCTYPE html>
<html lang="uk">
<head>
  <meta charset="UTF-8">
  <title>POST (then)</title>
</head>
<body>
  <button id="send">Відправити POST</button>
  <pre id="output"></pre>

  <script>
const btn = document.getElementById("send");
const output = document.getElementById("output");

btn.addEventListener("click", () => {
  fetch("https://dummyjson.com/products/add", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      title: "BMW Pencil",
      category: "stationery"
    })
  })
  .then(res => res.json())
  .then(data => {
    console.log("Created:", data);
    output.textContent = JSON.stringify(data, null, 2);
  })
  .catch(err => {
    console.error("Error:", err);
    output.textContent = "Помилка запиту";
  });
});
</script>

</body>
</html>
```

Надсилання POST-запиту

- У body має бути рядок JSON .
 - Завжди встановлюйте Content-Type: application/json у заголовках під час надсилання JSON.
- Аналогічний уривок коду для async/await:

```
try {
  const res = await fetch("https://dummyjson.com/products/add", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      title: "BMW Pencil",
      category: "stationery"
    })
  });

  const data = await res.json();

  console.log("Created:", data);
  output.textContent = JSON.stringify(data, null, 2);

} catch (err) {
  console.error("Error:", err);
  output.textContent = "Помилка запиту";
}
```

Грамотне поводження з помилками

```
<!doctype html>
<html>
<body>
<button id="run">GET 404</button>
<pre id="out"></pre>
<script>
const out = document.getElementById("out");

document.getElementById("run").addEventListener("click", () => {
  fetch("https://dummyjson.com/products/0")
    .then(res => {
      out.textContent =
        `fetch fulfilled ✅ \nstatus=${res.status}\nok=${res.ok}\n`;
      return res.json(); // тут dummyjson повертає JSON з описом помилки
    })
    .then(data => {
      out.textContent += "\nBody:\n" + JSON.stringify(data, null, 2);
    })
    .catch(err => {
      out.textContent += "\ncatch ❌ " + err;
    });
});
</script>
</body>
</html>
```

- `fetch fulfilled` ✅ – проміс виконався успішно.
 - `fetch()` НЕ переходить у `catch` автоматично.
 - `catch` у `fetch` спрацьовує зазвичай для: network errors (нема інтернету, DNS, CORS-блок тощо); помилки парсингу (`res.json()` впав, бо відповідь не JSON).

The screenshot shows a web browser at `127.0.0.1:5500/fetch-test.html`. The console displays the message `GET 404` and the `fetch fulfilled` status. The Network tab shows a failed request to `https://dummyjson.com/products/0` with a status of `404 Not Found`. The response headers are visible, including `Access-Control-Allow-Origin: *` and `Content-Type: application/json; charset=utf-8`.

Name	Value
Request URL	https://dummyjson.com/products/0
Request Method	GET
Status Code	404 Not Found
Remote Address	104.21.61.23:443
Referrer Policy	strict-origin-when-cross-origin
Access-Control-Allow-Origin	*
Cf-Cache-Status	DYNAMIC
Cf-Ray	9d3887668b0bef8d-WAW
Content-Type	application/json; charset=utf-8
Date	Wed, 25 Feb 2026 16:20:14 GMT
Etag	W/"2b-IQyQdFvJMIROTqQVG2bK4ASsAgg"

Грамотне поводження з помилками

```
async function fetchAddress() {  
  try {  
    const response = await fetch(addressUrl);  
    // Check if response is successful  
    if (!response.ok) {  
      throw new Error(`Network error. Status: ${response.status}`);  
    }  
    const addressData = await response.json();  
    console.log(addressData);  
  } catch (error) {  
    console.error("Error:", error);  
  }  
}  
  
fetchAddress();
```



- Якщо мережа дасть збій або URL-адреса API неправильна, це буде виявлено в блоці `.catch()`.
 - Завжди додавайте `.catch()` в кінець ваших `fetch`-ланцюжків, щоб безпечно обробляти помилки.
- Огляд коду:
 - 1 - блок коду `if(!response.ok)` перевіряє стан відповіді в діапазоні від 200 до 299; інакше він реєструє помилку.
 - 2 - у журналі при виконанні `throw new Error()`; записується спеціальне повідомлення, коли відповідь не вдається.
 - Блок `catch()` обробляє недійсні відповіді та проблеми з мережею.

Грамотне поводження з помилками

```
try {
  const response = await fetch('https://restcountries.com/v4.1/all');

  if (response.ok) {
    console.log('Promise resolved and HTTP status is successful');
    // ...do something with the response
  } else {
    // Custom message for failed HTTP codes
    if (response.status === 404) throw new Error('404, Not found');
    if (response.status === 500) throw new Error('500, internal server error');
    // For any other server error
    throw new Error(response.status);
  }
} catch (error) {
  console.error('Fetch', error);
  // Output e.g.: "Fetch Error: 404, Not found"
}
```

- Під час використання Fetch API можуть виникати різні помилки, такі як: помилка сервера (500), помилка «не знайдено» (404), помилка мережі, помилка CORS тощо.
 - коли `response.ok` повертає `false`, ми можемо просто викинути помилку, тож блок `catch` буде виконано.
 - `catch` потім оброблятиме всі типи помилок.
 - `catch` приймає аргумент, який можна налаштувати під час викидання помилок з блоку `try`.
 - `response.status` можна використовувати для перевірки поверненого коду стану HTTP для відображення налаштованих повідомлень у консолі (наприклад, 400, 404, 500...).

Скасування fetch-запиту

- На жаль, `fetch()` не має засобів API з коробки, щоб скасувати `fetch`-запит після його початку.
 - Для скасування запиту потрібен додатковий інструмент – `AbortController`, який підключається так:

```
// Step 1: instantiate the abort controller  
const controller = new AbortController();  
  
// Step 2: make the fetch() aware of controller.signal  
fetch(..., { signal: controller.signal });  
  
// Step 3: call to cancel the request  
controller.abort();
```

- а) перед запуском запиту створіть екземпляр контролера переривання: `controller = new AbortController()`.
- б) під час правильного запуску запиту використовуйте аргумент `options` `fetch(url, { signal: controller.signal })` та встановіть властивість `signal` на `controller.signal`.
- в) якщо вам потрібно скасувати запит, просто викличте метод `controller.abort()`.

```

<script>
let controller = null;
const out = document.getElementById("out");
const startBtn = document.getElementById("start");
const cancelBtn = document.getElementById("cancel");

startBtn.addEventListener("click", async () => {
  // якщо раптом вже йде запит — скасовуємо попередній
  if (controller) controller.abort();

  controller = new AbortController();
  cancelBtn.disabled = false;

  out.textContent =
    "Запит відправлено...\n" +
    "Порада: DevTools → Network → Throttling → Slow 3G, щоб встигнути Cancel.\n";

  try {
    const res = await fetch("https://dummyjson.com/products", {
      signal: controller.signal
    });

    const data = await res.json();
    out.textContent += "\n✅ Отримано (перші 2 товари):\n";
    out.textContent += JSON.stringify(data.products.slice(0, 2), null, 2);

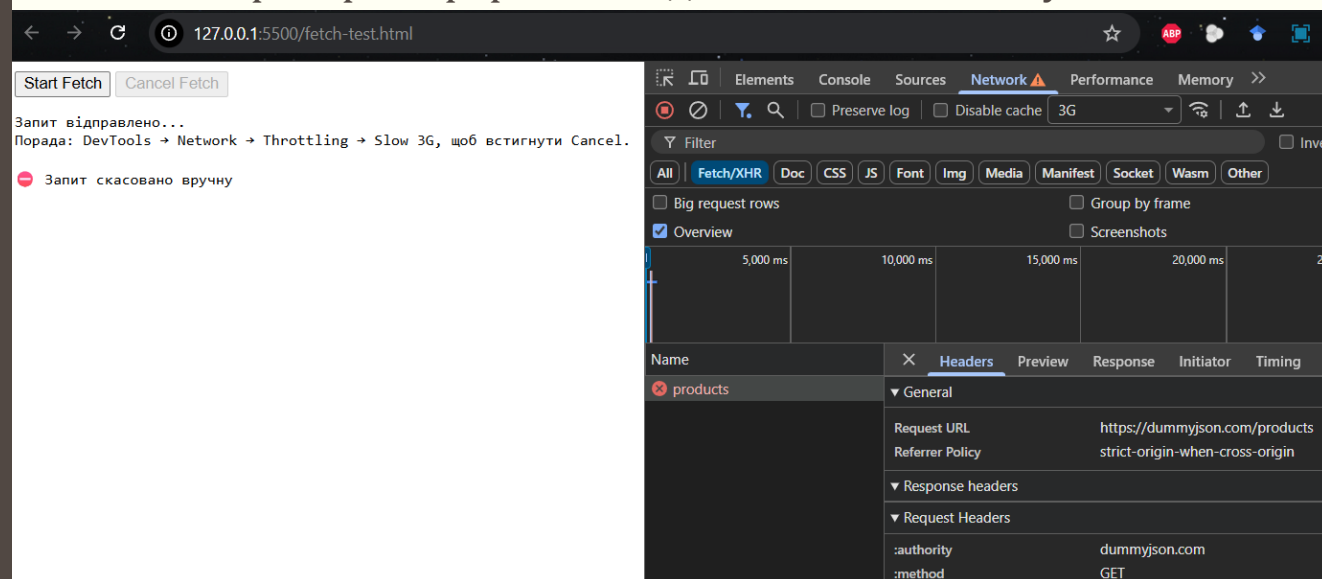
  } catch (err) {
    if (err.name === "AbortError") {
      out.textContent += "\n❌ Запит скасовано вручну";
    } else {
      out.textContent += "\n❌ Помилка: " + err.message;
    }
  } finally {
    controller = null;
    cancelBtn.disabled = true;
  }
});

cancelBtn.addEventListener("click", () => {
  if (controller) controller.abort();
});
</script>
</body>
</html>

```

Скасування fetch-запиту

- Штучно уповільнимо мережу в браузері.
 - Якщо запит вже виконується, ми його скасовуємо, - це захист від race conditions.
 - signal підключає fetch до controller, тепер controller.abort() може його зупинити.
 - Екземпляри контролера переривання не можна використовувати повторно: щоразу, коли ви запускаєте fetch-запит, потрібно створювати новий екземпляр контролера переривання для кожного запиту.



```

<script>
  async function fetchWithTimeout(resource, options = {}) {
    const { timeout = 8000 } = options;

    const controller = new AbortController();
    const id = setTimeout(() => controller.abort(), timeout);

    const response = await fetch(resource, {
      ...options,
      signal: controller.signal
    });

    clearTimeout(id);
    return response;
  }

  const out = document.getElementById("out");

  document.getElementById("run").addEventListener("click", async () => {
    out.textContent =
      "Запит з timeout=1000ms.\n" +
      "Щоб гарантовано побачити AbortError: DevTools → Network → Throttling → Slow  
3G + увімкни Disable cache.\n\n";

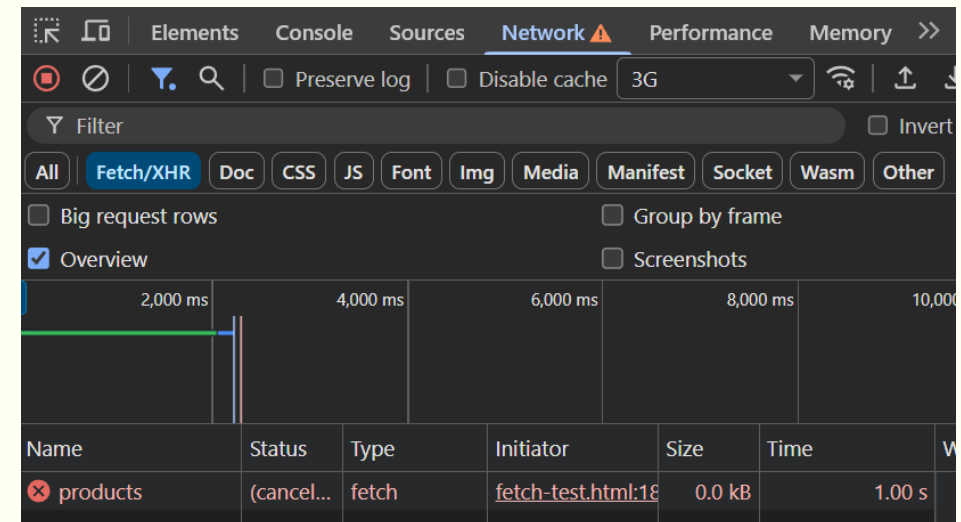
    try {
      const response = await fetchWithTimeout("https://dummyjson.com/products", {
        timeout: 1000
      });

      const data = await response.json();
      out.textContent += "✅ Успіх (перші 2 товари):\n";
      out.textContent += JSON.stringify(data.products.slice(0, 2), null, 2);
    } catch (error) {
      out.textContent += "❌ Помилка: " + error.name + "\n";
      out.textContent += "AbortError означає: запит перервано таймаутом.\n";
    }
  });
</script>
</body>
</html>

```

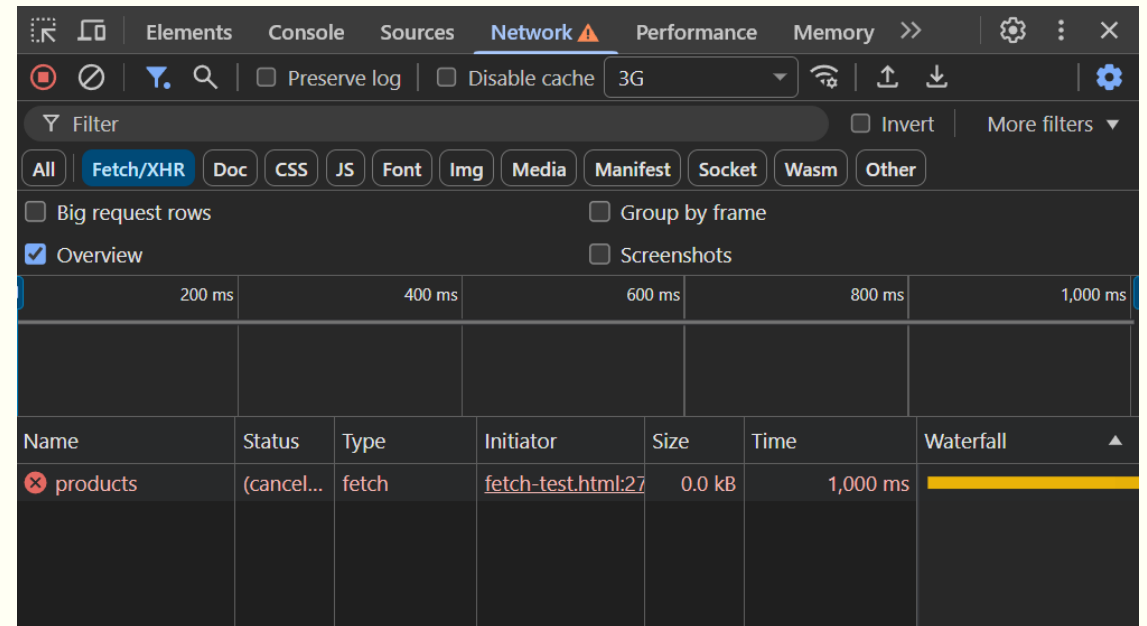
Час очікування fetch-запиту

- За замовчуванням час очікування fetch-запиту визначається браузером.
 - У Chrome час очікування мережевого запиту становить 300 секунд, а у Firefox – 90 секунд.
 - Ці значення набагато більші, ніж бажатиме очікувати користувач.
 - `const id = setTimeout(() => controller.abort(), timeout)` запускає функцію відліку часу.
 - Після закінчення часу `timeout`, якщо функція відліку часу не була очищена, `controller.abort()` скасовує `fetch`-запит.



Таймаут для fetch-запиту

- Поточні браузери підтримують простіший підхід, використовуючи `AbortSignal.timeout()`:



```

<!doctype html>
<html lang="uk">
<head>
  <meta charset="utf-8" />
  <title>Parallel Fetch (Promise.all)</title>
</head>
<body>

<button id="run">Load data (parallel)</button>
<pre id="out"></pre>

<script>
const out = document.getElementById("out");

document.getElementById("run").addEventListener("click", async () => {
  out.textContent = "Запити відправлено паралельно...\n";

  try {
    // Паралельні запити
    const [res1, res2] = await Promise.all([
      fetch("https://dummyjson.com/products/1"),
      fetch("https://dummyjson.com/products/2")
    ]);

    // Парсимо паралельно теж
    const [data1, data2] = await Promise.all([
      res1.json(),
      res2.json()
    ]);

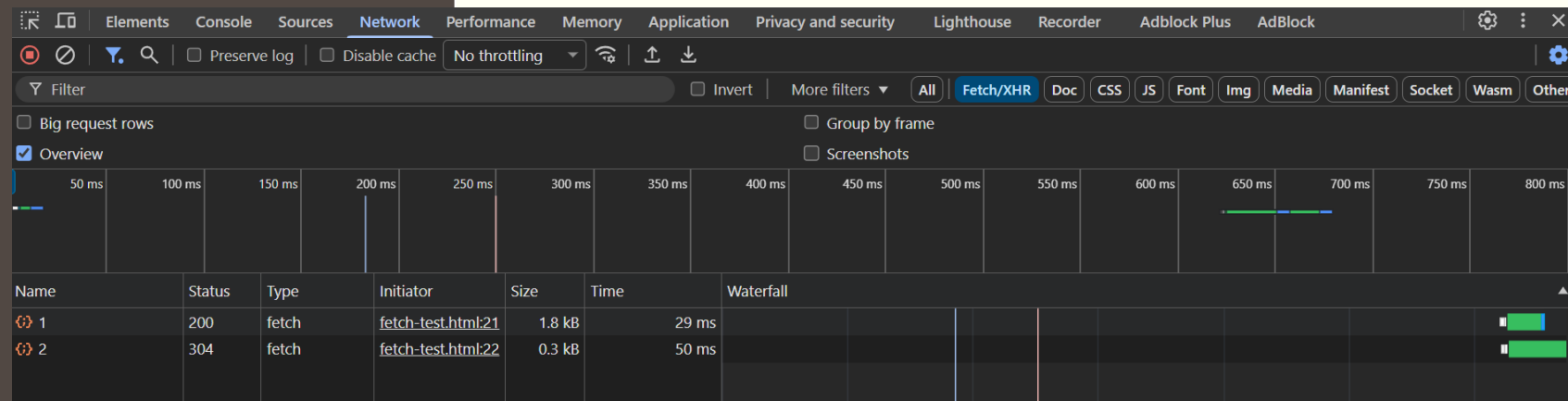
    out.textContent += "\n✅ Результат:\n";
    out.textContent += JSON.stringify(
      [data1.title, data2.title],
      null,
      2
    );

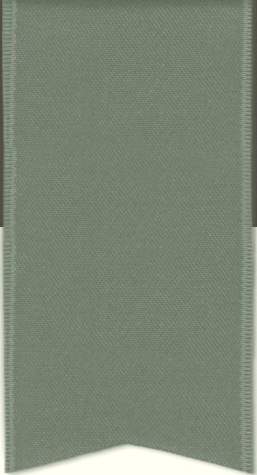
  } catch (err) {
    out.textContent += "\n❌ Помилка: " + err.message;
  }
});
</script>
</body>
</html>

```

Паралельні fetch-запити

- У стовпчику Waterfall видно паралельний запуск fetch-запитів.
 - Для виконання паралельних fetch-запитів використовуйте допоміжну функцію Promise.all().
 - Якщо будь-який запит завершується невдачею, то весь паралельний проміс одразу відхиляється з помилкою невдалого запиту.
 - Якщо ви хочете, щоб усі паралельні запити завершувалися, незважаючи на те, що будь-який з них зазнав невдачі, розгляньте можливість використання Promise.allSettled().





ДЯКУЮ ЗА УВАГУ!

Наступне питання: Тестування API за допомогою Postman.